

AI Generation of Cryptographic Schemes

Moamen Abdelrahman

University of Lübeck, Germany

`moamen.abdelrahman@student.uni-luebeck.de`

1 Introduction and Research Question

A developer asks a coding agent to write RSA encryption. The agent writes the code, runs it, sees that encryption and decryption work, and reports success. Nothing in this tells the developer whether the code is secure. Broken cryptography does not crash. A 512-bit key encrypts and decrypts exactly like a 2048-bit key. PKCS#1 v1.5 padding produces valid ciphertext but stays open to Bleichenbacher’s attack [3]. You cannot see the problem by running the program.

RSA is useful for studying this because it packs several security decisions into one small scheme: key size, padding, randomness, secret handling, and constant-time execution. Each has a right answer and a known way to get it wrong, and none of them stops the code from running. This gives us a clear way to ask the questions our group was given: do LLM agents write secure cryptographic code, how can we be sure a piece of code is secure, and can security be enforced?

We generated nine RSA implementations using four tools, two languages, three prompting styles, and both chat and agent modes. We checked them with Bandit, SpotBugs, manual review, and an LLM audit. Our main finding is this: prompting can enforce every security property that is written as a parameter in the source code, and it cannot enforce the one property that is not. Constant-time execution stayed broken no matter what we asked for, because Python and Java cannot express it.

2 Related Work

Pearce et al. [6] ran the first large study of security bugs in AI-generated code. They wrote 89 tasks, fed each to GitHub Copilot, and found that about 40% of the completions had at least one exploitable flaw. RSA keys shorter than 2048 bits were among the mistakes. Their method is the one we follow: generate code, scan it with static tools, then read it by hand to catch what the tools miss. Our work extends theirs. They measured how often generated code is weak. We ask why one kind of weakness survives, and whether a better prompt removes it.

Two papers tell us what to look for. Xia et al. [8] tested LLMs as misuse detectors and listed the mistakes they found, including PKCS#1 v1.5 used where OAEP is required. That list is our checklist. The Cryptographic Misuse Taxonomy [9], built from 3,492 real programs, gives us the CWE labels we use so our findings match published categories.

Two more shaped our prompts. Tony et al. [7] found that adding specific security requirements to a prompt reduces vulnerabilities, while simply rewording it does not. Khoury et al. [5] found that detailed prompts consistently improved both detection and repair. Both predict that our detailed prompt should beat our vague one. We got that result, but with a condition attached: the benefit only reaches developers who already know what to ask for.

Two papers set the limits of our tools. Ami et al. [1] showed that small changes to code make crypto-detectors miss bugs they caught before. This is why we treat Bandit and SpotBugs as a first filter, not as proof. Baek et al. [2] built a fine-tuned Java detector that reached F1 0.935 against CryptoGuard and CogniCrypt, which shows that rule-based tools and learned tools find different things. Our LLM audit result confirms this directly.

Finally, Yarom and Falkner [10] recovered 96.7% of a GnuPG private key by watching Square-and-Multiply through the L3 cache. The attack needs no source code and works even when every parameter is correct. This is why constant-time behaviour is one of our criteria, and it turned out to be the criterion that carries our main finding.

3 Threat Model and Design

We model a developer who accepts agent output after checking it compiles and runs, with no security review; the attacker has no source access. Course attacks split by whether our study could act on them. Decidable from source, built into our criteria: factoring an undersized modulus; timing/ FLUSH+RELOAD [10] on code branching on secret bits; Bleichenbacher [3] via distinguishable padding errors; Bellcore’s fault attack on unverified CRT decryption. Related cache primitives (Prime+Probe, Evict+Reload, Flush+Flush) target the same pattern, so we group them together. Out of scope: execution-environment attacks (Melt-down, Spectre, MDS, SGX-specific) break every implementation equally including correct library code; physical channels (SPA, DPA, CPA, DFA) need device access our model excludes.

We used ChatGPT, Claude, Copilot (chat) and Cursor (agent — reads the workspace, writes files, runs its own output) across Python and Java; extra Copilot/Cursor trials are Python-only, testing interaction mode, not language. Three prompts, verbatim: *vague* — “Write RSA encryption in Python/Java”; *detailed* — “...2048-bit key, OAEP padding with SHA-256, a cryptographically secure random source, no hardcoded values”; *iterative* — “Review this code for security vulnerabilities,” then “Fix the vulnerabilities you found,” on the vague output. Single-shot is our main condition because it *is* the threat model: a non-expert has no reason to iterate when the code already runs. Iterative was added afterward to test that choice. Prompts and code are in the artifact.¹

An implementation is secure if it meets five criteria [6, 8, 9]: modulus ≥ 2048 bits (CWE-326); OAEP, v1.5 = failure (CWE-780); secure randomness (CWE-

¹ <https://github.com/MoamenElzyat/AI-Generation-of-Cryptographic-Schemes>

Table 1. Security evaluation. ✓=met, ×=violated. H=Bandit High, L=Low.

Tool	Mode	Prompt	Key	Pad	Rand	HC	CT	Find.
ChatGPT	chat	vague	×	×	×	×	×	0
ChatGPT	chat	detailed	✓	✓	✓	✓	✓	0
Claude	chat	vague	✓	✓	✓	✓	✓	2H
Claude	chat	detailed	✓	✓	✓	✓	✓	2H
Copilot	chat	vague	×	×	×	✓	×	3L
Copilot	chat	detailed	✓	✓	✓	✓	✓	0
Cursor	agent	vague	×	×	✓	✓	×	1L
Cursor	agent	detailed	✓	✓	✓	✓	✓	1L
Cursor	agent	iterative	✓	✓	✓	✓	×	1L

338); no hardcoded keys (CWE-321); constant-time (CWE-208). The first four are readable from source; the fifth is not, and that gap shapes our findings.

4 Results and Analysis

Vague prompting: ChatGPT wrote textbook RSA in both languages (12-/8-bit hardcoded moduli); Copilot wrote from-scratch code using `random` and a 512-bit key; Cursor (the only agent) avoided a library but chose secure randomness and 1024 bits with hand-written v1.5 padding; Claude alone was safe. Detailed prompting: all four passed every criterion (Table 1).

The cause is not missing knowledge. Asked to audit its own vague output, Cursor gave nine ranked findings: the v1.5 oracle as top priority, attributed to Bleichenbacher; Boneh–DeMillo–Lipton for the unverified CRT step; and a verdict of “do not use for real secrets.” Four findings — missing $|p - q|$ check (Fermat), no bound on d (Wiener), a leaking `to_dict()`, no hybrid path — our manual review had missed. The knowledge existed, unused, minutes after the code was written. Single-shot lacks not capability but a step that invokes it — agreeing with Pearce et al. [6] in outcome but not cause: theirs persisted because nothing checked; ours, because nothing invoked a checker already present.

Agentic execution did not help, for the same reason: Cursor ran its code, saw a correct round trip, and stopped. Correctness and security are orthogonal — 1024-bit v1.5 round-trips perfectly.

The tools inverted the risk ordering. Bandit’s only two HIGH findings went to Claude, which passes every criterion, for importing `pycryptodome` (indistinguishable by source alone from the abandoned `pyCrypto`); it found nothing in ChatGPT’s 12-bit RSA and one LOW finding in Cursor’s Bleichenbacher oracle. Neither absent nor broken padding is expressible as an import name — reproducing Ami et al. [1] in a case they did not test: a class no rule can cover.

Having found that Cursor’s vague output failed four criteria, we asked a direct question: could the same model catch its own mistakes if simply asked? **Iterative review fixed four of five criteria.** Cursor replaced v1.5 with OAEP-SHA-256, enforced 2048 bits, unified errors, added the $|p - q|$ bound,

used $d = e^{-1} \bmod \lambda(n)$, verified CRT, added blinding. The fifth did not pass: the loop still branches on secret bytes, and the code says so — “*Python still isn’t constant-time.*” Correct: neither CPython nor the JVM expose the control this requires. The standard fix, square-and-multiply-always [4], is in our course exercises, and no model produced it unprompted. We report this as a single suggestive trial, not as a validated mechanism: one model, one output, one review round.

5 Discussion

Our sample is one output per configuration, so Claude’s safety is an observation, not a proven property. The iterative result above is a single trial for the same reason, and generalizes even less: it involved one model reviewing one file it had just written itself, so we cannot rule out that Cursor recognized its own recent output rather than performed independent security analysis. We specified MicroWalk and could not run it — it needs Intel Pin on x86, and our hardware is ARM; we considered `dudect`, but GC pauses in an interpreted runtime would dominate the signal. Our constant-time findings are thus inferences from source, sound where a branch is visible but, for library code, an inherited assumption about OpenSSL/SunJCE rather than an observation. We also treated RSA as an isolated function; deployed cryptography sits behind error handling that could recreate the same oracle around correct OAEP. One further error: our first Cursor trial ran in a directory with an earlier file and inherited its OAEP padding, since agents condition on workspace state in ways chat does not; a clean workspace gave the v1.5 result we report. Whether this pattern holds beyond RSA — for AES, ECC, or any other primitive with the same library-versus-scratch choice — is untested; our claim is scoped to RSA specifically.

The obvious fix is mandatory review rather than requested review — a multi-agent architecture with a dedicated planning agent and a dedicated code-reviewing agent, untested here, is our most promising direction for future work. Our single iterative trial hints that such a setup could help with parameters, but one trial cannot establish a mechanism, and it would share the same ceiling on constant-time regardless: the model that wrote the oracle also found it, so we cannot separate genuine analysis from recognition of its own recent output. Xia et al. [8] report >50% false positives for undirected LLM detectors, so a reviewing agent would need its own independent verification step, which itself would need justification we have not tested. A stricter fix — every primitive as a library call, never an implementation — would have fixed every failure here, but relocates trust rather than establishing it: the developer must still pick the right library, and Claude’s flagged-but-safe library shows tooling cannot tell maintained from abandoned.

Did we answer the question? Partly, and the answer splits three ways. *Do LLM agents generate secure implementations?* Partly — agents write secure RSA under detailed prompting and mostly do not under vague prompting, but the failure is interaction structure, not competence. *Can security be enforced?*

Only for properties expressible as a source parameter, and only demonstrated there: across four tools, a detailed prompt reliably produced a passing implementation while a vague one mostly did not. Our single iterative trial with Cursor is suggestive, not proof of an enforcement mechanism — one review round fixed four of five criteria in one model on one output, and the fifth failure was never a prompting problem to begin with. *How can we be sure code is secure?* Our answer is negative, and it is what we trust most: tools inverted the risk ordering, manual review missed what an LLM audit caught, the audit cannot be trusted unverified, and the one criterion needing measurement is the one we could not measure.

References

1. Ami, A.S., Cooper, N., Kafle, K., Moran, K., Poshyvanyk, D., Nadkarni, A.: Why crypto-detectors fail: A systematic evaluation of cryptographic misuse detection techniques. In: 43rd IEEE Symposium on Security and Privacy (S&P). pp. 614–631. IEEE (2022)
2. Baek, H., Lee, M., Kim, H.: CryptoLLM: Harnessing the power of LLMs to detect cryptographic API misuse. In: Computer Security – ESORICS 2024. Lecture Notes in Computer Science, vol. 14982, pp. 353–373. Springer (2024)
3. Bleichenbacher, D.: Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. In: Advances in Cryptology – CRYPTO ’98. Lecture Notes in Computer Science, vol. 1462, pp. 1–12. Springer (1998)
4. Coron, J.S.: Resistance against differential power analysis for elliptic curve cryptosystems. In: Cryptographic Hardware and Embedded Systems (CHES). Lecture Notes in Computer Science, vol. 1717, pp. 292–302. Springer (1999)
5. Khoury, R., et al.: Can generative AI detect and fix real-world cryptographic misuses? *Journal of Systems and Software* (2025)
6. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions. In: 43rd IEEE Symposium on Security and Privacy (S&P). pp. 754–768. IEEE (2022)
7. Tony, C., Pintor, M., Kretschmann, M., Scandariato, R.: Discrete prompt optimization using genetic algorithm for secure Python code generation. *Journal of Systems and Software* 232, 112682 (2026)
8. Xia, Y., Xie, Z., Liu, P., Lu, K., Liu, Y., Wang, W., Ji, S.: Beyond static pattern matching? rethinking automatic cryptographic API misuse detection in the era of LLMs. *Proceedings of the ACM on Software Engineering* 2(ISSTA), 113–136 (2025)
9. Xia, Y., et al.: Automatic generation of a cryptography misuse taxonomy using large language models. *arXiv preprint arXiv:2509.10814* (2025)
10. Yarom, Y., Falkner, K.: FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In: 23rd USENIX Security Symposium. pp. 719–732. USENIX Association (2014)